



Error Handling

Занятие 15 • Инструктор — Андрей Помелов

Разбираемся, как писать надёжный код, который не ломается при неожиданных ситуациях.



Что такое ошибка?

Ошибка — это ситуация, при которой программа **не может выполнить действие корректно**. Например, обратиться к несуществующему файлу, получить неожиданный тип данных или потерять соединение с сервером.

⚡ Runtime Error

Возникает во время выполнения программы

abc Syntax Error

Неправильно написанный код

🌀 Logic Error

Программа работает, но даёт неверный результат



ЗАЧЕМ ЭТО НУЖНО

Обработка ошибок — это защита программы

Без обработки ошибок программа просто **«падает»** — пользователь видит белый экран или необъяснимый сбой. Грамотная обработка позволяет:

- Продолжить работу при возникновении проблемы
- Сообщить пользователю понятное сообщение
- Записать ошибку в лог для отладки

Где чаще всего возникают ошибки?

В реальных приложениях есть несколько «горячих точек», где ошибки появляются особенно часто.

Запросы к API

Сервер недоступен, превышен таймаут, ошибка 404 или 500

Некорректный JSON

Ответ пришёл, но не является валидным JSON-объектом

Проблемы сети

Пользователь офлайн или соединение прерывается

Неверные данные

Пользователь ввёл данные не того типа или в неправильном формате

Конструкция **try-catch**

Блок `try-catch` — основной инструмент перехвата ошибок в JavaScript. Он позволяет «обернуть» опасный код и обработать любую возникшую ошибку.

```
try {  
  // Код, который может вызвать ошибку  
  const data = JSON.parse(userInput);  
  console.log(data);  
} catch (error) {  
  // Что делать, если ошибка произошла  
  console.error('Ошибка разбора JSON:', error.message);  
}
```

 Блок `catch` получает объект ошибки с полями `message`, `name` и `stack`.

Как работает **try-catch**



Если ошибки нет — блок `catch` пропускается. Опциональный блок `finally` выполняется **всегда**, независимо от результата — удобно для закрытия соединений или очистки ресурсов.

THROW NEW ERROR

Зачем выбрасывать ошибки вручную?

Иногда нужно **явно сообщить** об ошибке, даже если JavaScript сам её не обнаружил. Например, функция получила `null` вместо числа — это логическая проблема, которую стоит зафиксировать.

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error('Деление на ноль!');  
  }  
  return a / b;  
}
```

⚠️ Используй `throw`, чтобы сделать контракт функции явным и понятным для других разработчиков.

Почему это важно

Ошибка будет поймана ближайшим `catch`

- Сообщение станет информативным
- Код станет предсказуемым



Практика: обработка ошибок в Axios

Запросы через **Axios** часто завершаются ошибкой — сервер может быть недоступен или вернуть код 4xx/5xx. Вот типичный паттерн обработки:

```
import axios from 'axios';

async function fetchUser(id) {
  try {
    const response = await axios.get(`/api/users/${id}`);
    return response.data;
  } catch (error) {
    if (error.response) {
      // Сервер ответил, но с ошибкой
      console.error('Ошибка сервера:', error.response.status);
    } else if (error.request) {
      // Запрос ушёл, но ответа нет
      console.error('Нет ответа от сервера');
    } else {
      console.error('Ошибка запроса:', error.message);
    }
  }
}
```

Итог: что мы узнали

Обработка ошибок делает программу **устойчивой** и позволяет контролировать проблемные ситуации вместо того, чтобы просто «падать».

1

Понять природу ошибок

Ошибки неизбежны — важно их предвидеть

2

Использовать try-catch

Перехватывать и обрабатывать все опасные операции

3

Выбрасывать throw

Явно сигнализировать о логических проблемах

4

Применять на практике

Особенно при работе с API и внешними данными

✓ Надёжный код — это не тот, в котором нет ошибок, а тот, который умеет с ними справляться.